

Code-Layout, Readability and Reusability

Date	July 7, 2026
Team ID	XXXXXX
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 styling conventions strictly adhered to across all .py files.
2	Proper File Structure	Files and folders are logically organized	Yes	Explicitly decoupled into / frontend (Streamlit) and / backend (FastAPI).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Clear descriptive naming schemas applied (e.g., <code>extracted_theme</code> , <code>user_metrics_log</code>).
4	Function / Method Names	Functions are descriptively named	Yes	Explicit naming conventions utilized (e.g., <code>fetch_wikipedia_facts()</code> , <code>generate_icebreakers()</code>).
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings included for all API endpoints and pipeline transformation steps.
6	Modular Design	Code is split into reusable functions/modules	Yes	ML inference models and storage handlers isolated into individual helper utilities.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Cleaned and refactored using code linters to eliminate redundant loops and operations.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Managed via HTTPExceptions for endpoints and try-except wrappers for live Wikipedia API requests.

Reusable Components / Modules:

S.No	Component / Module Name	Language/Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Wikipedia Client Wrapper	Python	Injected across multiple event validation workflows to fetch live real-time contexts.	High

2	JSON Metric Handler	Python	Used by multiple session trackers to log and append thumbs up/down user interactions locally.	High
3	DistilBERT Theme Extractor	Python / HuggingFace	Shared module utilized to parse text topics from diverse external input fields.	Medium
4	Streamlit Tab UI Builder	Python / Streamlit	Standardized multi-tab functional container framework used repeatedly across the frontend layout.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Excellent separation of concerns between backend routing logic and frontend presentation components.
Readability	5	Clear PEP 8 compliant formatting makes the codebase easy to read and understand for any onboarding developer.
Reusability	4	API integrations and local JSON persistence utility modules are built generic enough for external system imports.
Documentation / Comments	5	Thorough documentation covers environment setups, dependency chains, and inline business logic.
Overall Score	4.75/5	Robust, clean, and highly professional architecture aligning with modern AI-driven application practices.